Arsitektur Dokumentasi Skema Otomatis dan Pelacakan Data Lineage Tanpa Eksposur Kredensial pada Lingkungan Enterprise Data Management (EDM) Perbankan

Modernisasi infrastruktur data di sektor perbankan telah menggeser paradigma penyimpanan dari basis data transaksional (*Online Transaction Processing* atau OLTP) yang terfragmentasi menuju gudang data analitik berbasis komputasi awan (*Online Analytical Processing* atau OLAP) seperti Google BigQuery[1]. Dalam arsitektur *pipeline* yang mengalirkan data dari berbagai mesin basis data produksi OLTP menuju lapisan *Data Raw* hingga *Data Mart* di BigQuery, ketiadaan dokumentasi *Entity-Relationship Diagram* (ERD) yang terotomatisasi serta kaburnya silsilah data (*data lineage*) menciptakan risiko operasional dan kepatuhan yang signifikan[3]. Ketika tabel baru ditambahkan atau definisi kolom dimodifikasi pada lapisan hulu, tim rekayasa data dan analis perbankan kerap kehilangan visibilitas terhadap dampak hilir (*downstream impact*), yang berujung pada rusaknya laporan regulasi maupun dasbor eksekutif[5]. Namun, di lingkungan perbankan yang tunduk pada regulasi keamanan siber dan tata kelola privasi yang ketat, solusi otomatisasi dokumentasi tidak dapat diimplementasikan dengan cara konvensional yang menyimpan kredensial basis data produksi di dalam aplikasi pemodelan atau menyerahkannya kepada layanan pihak ketiga[2].

Analisis arsitektur ini membedah secara komprehensif lima aspek fundamental dalam merancang sistem dokumentasi ERD otomatis dan pelacakan *data lineage* berbasis kontainer Docker yang mematuhi prinsip *zero-credential-exposure* (tanpa eksposur kredensial produksi). Setiap bagian dievaluasi secara berurutan dengan merujuk langsung pada dokumentasi teknis resmi, struktur kode sumber terbuka (*open-source*), mekanisme kontrol akses *Identity and Access Management* (IAM), serta studi kasus nyata dari institusi perbankan digital berskala besar.

1. Alternatif Tool Dokumentasi ERD dan Skema Tanpa Koneksi atau Kredensial Live (Selain ChartDB dan DBML/dbdocs.io)

Selain ChartDB (https://github.com/chartdb/chartdb) dan ekosistem berbasis DBML, terdapat tiga perangkat lunak dokumentasi skema dan pemodelan ERD yang secara arsitektural mendukung metode input statis murni—baik melalui penempelan kueri struktur (*paste-query*), pengunggahan berkas *dump Data Definition Language* (DDL) statis, pembacaan berkas skema *Object-Relational Mapping* (ORM), maupun impor berkas migrasi SQL—tanpa memerlukan

koneksi jaringan langsung ataupun kredensial ke basis data produksi[8]. Sesuai dengan prinsip *zero-credential-exposure*, fitur koneksi basis data langsung yang mungkin tersedia sebagai jalur lain pada beberapa perangkat tersebut diabaikan sepenuhnya dalam kajian ini, dan evaluasi difokuskan murni pada mekanisme pemrosesan berkas statisnya[9].

A. Azimutt (Mode Sumber Statis: SQL DDL, AML, JSON, dan Prisma)

Perangkat pertama dengan nama persis **Azimutt** memiliki dokumentasi resmi sumber input (*Sources*) di https://azimutt.app/docs/sources[9], panduan pembuatan proyek di https://azimutt.app/docs/create-your-project[12], dokumentasi manajemen tata letak (*Layouts*) di https://azimutt.app/docs/layouts[13], dokumentasi anotasi *Memos* di https://azimutt.app/blog/document-your-database-with-memos[14], serta kode sumber *template* dokumentasi resmi di repositori GitHub https://github.com/azimuttapp/azimutt/blob/main/backend/lib/azimutt_web/templates/website/docs/create-your-project.html.heex[15] dan https://github.com/azimuttapp/azimutt/blob/main/backend/lib/azimutt_web/templates/website/docs/badge.html.heex[16]. Azimutt merupakan penjelajah skema basis data *open-source* yang dapat di-*self-host* secara mandiri dan dirancang khusus untuk menangani basis data dunia nyata berskala masif (ratusan tabel dengan ratusan kolom) yang kerap membuat perangkat ERD konvensional mengalami kegagalan *rendering* atau kelebihan beban informasi[10].

Untuk beroperasi tanpa menyentuh kredensial produksi, Azimutt menyediakan empat jalur input statis yang seluruhnya dikonversi menjadi struktur data internal di sisi klien (*frontend*)[9]. Jalur pertama adalah **SQL source** (DDL *dump* statis), di mana administrator mengekspor struktur skema basis data menjadi berkas .sql yang memuat instruksi CREATE TABLE, CREATE INDEX, COMMENTS, serta definisi *foreign keys*[9]. Berkas tersebut diunggah dari komputer lokal atau dimuat melalui URL statis (seperti https://azimutt.app/create?sql=$url\&name$ = project_name)[9]. Dokumentasi resmi Azimutt menegaskan bahwa seluruh proses *parsing* SQL dilakukan sepenuhnya di sisi *frontend* (aplikasi peramban web berbasis *Single-Page Application* / Elm), sehingga tidak ada skema maupun data bisnis yang dikirimkan ke peladen eksternal[12]. Jalur kedua adalah **JSON source**, yang di dalam arsitektur internal Azimutt berstatus sebagai sumber kebenaran utama (*ultimate truth*) karena seluruh format input lain pada akhirnya diterjemahkan ke dalam skema JSON ini sebelum divisualisasikan[18]. Apabila bank mengoperasikan mesin basis data OLTP spesifik yang dialek SQL-nya rumit atau berasal dari entitas ORM khusus, tim internal dapat menjalankan skrip ekstraksi metadata mandiri di lingkungan tertutup, memformat keluarannya mengikuti spesifikasi *Azimutt JSON Schema*, lalu mengimpor berkas .json statis tersebut ke dalam Azimutt[6]. Jalur ketiga adalah **AML (Azimutt Markup Language) source**, yakni bahasa pemodelan tekstual khusus yang dikembangkan oleh Azimutt (lengkap dengan pustaka mandiri dan ekstensi VS Code) untuk mendesain skema baru atau memperbarui dokumentasi skema secara deklaratif tanpa koneksi basis data[6]. Jalur keempat adalah **Prisma source**, yang mendukung pengunggahan langsung berkas definisi skema schema.prisma lokal maupun jarak jauh untuk menghasilkan diagram relasi secara instan[12].

Implikasi arsitektural Azimutt bagi lingkungan EDM perbankan terletak pada kapabilitas fitur

Layouts (https://azimutt.app/docs/layouts). Pada basis data inti perbankan (*core banking*) yang memiliki ratusan tabel, menampilkan seluruh entitas dalam satu kanvas tunggal justru menghilangkan nilai keterbacaan. Azimutt membatasi tampilan awal setiap entitas hanya pada 15 atribut terpenting (diurutkan berdasarkan *primary key*, relasi, indeks, lalu urutan SQL) dan memungkinkan arsitek data membagi satu skema besar menjadi banyak *layout* parsial yang diorganisasikan dalam struktur folder menggunakan karakter garis miring (contoh: teams / revenue / mrr). Setiap *layout* dapat diperkaya dengan pengelompokan visual berwarna di belakang tabel (**Groups**) serta elemen dokumentasi kontekstual berbasis *Markdown* (**Memos**) untuk menjelaskan logika bisnis atau cakupan tim tanpa mengubah struktur fisik basis data (https://azimutt.app/blog/document-your-database-with-memos)[13].

B. DrawSQL (Mode Impor Dump DDL Statis, ORM, dan REST API)

Perangkat kedua dengan nama persis **DrawSQL** mendokumentasikan mekanisme kerjanya secara resmi melalui panduan impor SQL DDL di https://drawsql.app/docs/import-from-ddl[11], panduan impor skema umum di https://drawsql.app/docs/import-your-schema[19], panduan impor skema Rails di https://drawsql.app/docs/import-rails-schema, halaman konversi SQL ke ERD di https://drawsql.app/sql-to-erd[20], ikhtisar kanvas /draw di https://drawsql.app/docs/draw-overview[21], alat pemformat SQL di https://drawsql.app/tools/sql-formatter, serta dokumentasi pengembang dan referensi REST API di https://drawsql.app/developers[22] dan https://drawsql.app/developers/api/reference[23].
DrawSQL merupakan platform visualisasi ERD yang secara desain tidak pernah meminta koneksi langsung ke basis data produksi, melainkan bekerja sepenuhnya melalui ekstraksi berkas definisi struktur statis[11].
Alur kerja tanpa kredensial pada DrawSQL diawali dengan ekstraksi *dump* hanya-struktur (*structure-only dump*) di lingkungan internal bank menggunakan utilitas bawaan DBMS untuk menghasilkan berkas teks yang hanya berisi pernyataan CREATE TABLE tanpa menyertakan baris data transaksi. Pada MySQL atau MariaDB, administrator mengeksekusi perintah mysqldump --no-data --skip-comments -u USERNAME -p DATABASE > schema.sql atau menjalankan SHOW CREATE TABLE table_name; untuk satu tabel spesifik. Pada PostgreSQL, ekstraksi dilakukan dengan perintah pg_dump --schema-only --no-owner --no-privileges DATABASE > schema.sql. Pada Microsoft SQL Server, ekstraksi dilakukan melalui *SQL Server Management Studio* (*Tasks -> Generate Scripts* dengan opsi *Types of data to script* diatur ke *Schema only* dan setiap pernyataan wajib dipisahkan dengan karakter titik koma atau *semicolon*). Lebih jauh lagi, dokumentasi resmi DrawSQL (https://drawsql.app/docs/import-from-ddl) menyediakan prosedur generasi skema yang bahkan tidak perlu menyentuh peladen basis data sama sekali, yakni dengan mengekstrak DDL langsung dari kode ORM aplikasi di repositori Git menggunakan perintah php artisan schema:dump pada Laravel (yang menulis ke database/schema/), perintah npx prisma migrate diff --from-empty --to-schema-datamodel prisma/schema.prisma --script pada Prisma, atau mengimpor berkas db/structure.sql serta db/schema.rb pada Ruby on Rails (https://drawsql.app/docs/import-rails-schema).
Apabila berkas *dump* tidak terformat dengan rapi, DrawSQL menyediakan pemformat SQL berbasis peramban (https://drawsql.app/tools/sql-formatter) yang memproses teks secara lokal

di sisi klien tanpa mengunggah data apa pun ke peladen. Saat teks SQL ditempelkan atau diunggah melalui menu *File -> Import*, parser secara otomatis mendeteksi dialek DBMS dari sintaksisnya—sebagai contoh, tipe data SERIAL dan JSONB dikenali sebagai PostgreSQL, sementara NVARCHAR dan IDENTITY dikenali sebagai SQL Server (https://drawsql.app/sql-to-erd)[24]. Parser tersebut mengekstrak tabel, kolom, tipe data, *nullability*, nilai *default*, *primary keys*, *unique constraints*, *indexes*, nilai *enum*, serta komentar tabel dan kolom (untuk MySQL dan PostgreSQL), lalu menggambar *foreign keys* sebagai garis relasi antar-tabel[11]. Pernyataan non-struktur seperti *views*, *triggers*, *stored procedures*, dan pernyataan INSERT dilewati (*skipped*) oleh parser, namun tetap dihitung terhadap batas ukuran teks parser sehingga ekspor *schema-only* sangat diwajibkan. Selain antarmuka web (yang pada kanvas bebas /draw menyimpan draf di localStorage peramban dengan batas 20 tabel), DrawSQL menyediakan REST API (POST https://api.drawsql.app/v1/diagrams) dan *Model Context Protocol* (MCP) server yang menerima *payload* string ddl (maksimal 200KB) atau objek schema patch (maksimal 500KB) dengan batas laju 10 permintaan per menit untuk menghasilkan diagram secara terprogram dari *pipeline* CI/CD (https://drawsql.app/developers/api/reference)[21].

C. SchemaLens (Jalur Impor Berkas Migrasi SQL Statis)

Perangkat ketiga dengan nama persis **SchemaLens** mendokumentasikan arsitektur tanpa kredensialnya melalui laman perbandingan teknis resmi di https://schemalens.net/compare/azimutt-alternative[10]. Berfokus pada dokumentasi referensi skema PostgreSQL bagi tim rekayasa, SchemaLens menyediakan jalur arsitektur spesifik bernama **Migration-import path** yang dirancang khusus untuk lingkungan integrasi berkelanjutan (*Continuous Integration* / CI) dan peninjauan kode[26]. Melalui jalur ini, SchemaLens membangun referensi skema langsung dari berkas-berkas migrasi SQL statis (*SQL migration files*) di dalam repositori kode tanpa memerlukan kredensial basis data sama sekali (*no database credentials at all*)[26]. Sistem ini melacak kebaruan dokumentasi secara otomatis melalui penanda waktu publikasi (*published timestamps* dan *reruns*) untuk memastikan apakah referensi skema mencerminkan kondisi hari ini atau bulan lalu, sekaligus memanfaatkan pemrosesan AI di atas struktur DDL tersebut untuk menghasilkan penjelasan skema menyeluruh, panduan *onboarding*, dan laporan risiko skema tanpa pernah menyentuh baris data tabel[26].

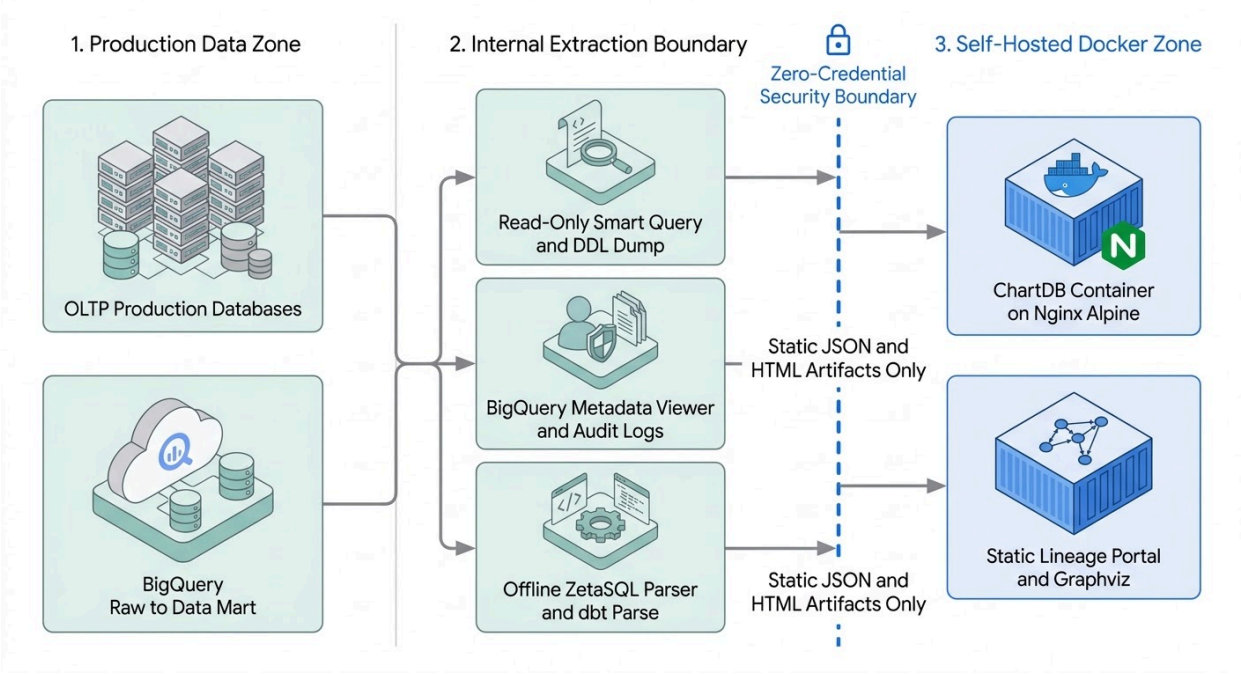
Nama Perangkat Lunak	Tautan Dokumentasi Resmi	Format Input Statis (Tanpa Kredensial)	Lokasi Pemrosesan & Opsi Self-Hosting	Dukungan Dialek & Kapabilitas Khusus EDM
Azimutt	https://azimutt.app/docs/sources	Berkas SQL DDL (CREATE TABLE), AML, *Azimutt JSON*	Parsing dilakukan di sisi *frontend* (SPA peramban);	Multi-DBMS; sanggup menangani ratusan tabel

		Schema, dan schema.prisma [6].	mendukung *self-hosting* penuh (*open-source*)[10].	dengan fitur *Layouts* parsial, *Groups*, dan anotasi *Markdown Memos*[10].
DrawSQL	https://drawsql.app/docs/import-from-ddl	Dump SQL DDL (*schema-only*), DBML, db/schema.rb (Rails), DrawSQL JSON, dan *payload* REST API (ddl maks 200KB)[11].	Pemformatan SQL dan penyimpanan draf /draw berjalan di localStorage peramban; layanan utama berbasis *cloud*[11].	MySQL, MariaDB, PostgreSQL, SQL Server (termasuk ekstraksi *offline* dari Laravel, Prisma, dan Rails).
SchemaLens (*Migration Path*)	https://schemalens.net/compare/azimutt-alternative	Berkas migrasi SQL statis (*SQL migration files*) dari repositori kode[26].	Layanan terkelola (*hosted*) yang memproses berkas migrasi SQL dari *pipeline* CI/CD tanpa koneksi DB[26].	Spesifik PostgreSQL; menyediakan *freshness timestamps* dan pembuatan dokumentasi risiko otomatis[26].

2. Evaluasi Performa Rendering dan Mekanisme Segmentasi Platform ERD Multi-Platform

Dalam lingkungan *Enterprise Data Management* (EDM) perbankan, skema basis data produksi lazimnya terdiri atas ratusan tabel yang saling berelasi lintas platform (seperti PostgreSQL, MySQL, dan Microsoft SQL Server) dengan puluhan hingga ratusan atribut pada setiap entitasnya[7]. Memvisualisasikan skema berskala masif ini ke dalam satu kanvas tunggal tanpa manajemen beban *Document Object Model* (DOM) dan mekanisme segmentasi yang terukur akan mengakibatkan kelumpuhan antarmuka peramban (*browser crash*) serta kelebihan beban informasi (*information overload*) bagi para arsitek data[10]. Oleh karena itu, evaluasi arsitektural terhadap platform pemodelan data modern—yakni **Azimutt**, **Eraser.io**, dan **DrawSQL**—difokuskan pada tiga dimensi teknis: efisiensi *rendering* kanvas, mekanisme

manajemen segmentasi skema multi-tim, serta alur otomatisasi sinkronisasi skema secara *zero-credential-exposure*[9].



A. Efisiensi DOM dan Rendering Canvas pada Skema Berskala Ratusan Tabel

Perbedaan paling mendasar antara platform ERD generasi baru dan perangkat pemodelan kanvas datar (*flat canvas*) tradisional terletak pada strategi pengendalian jumlah simpul DOM yang dirender secara simultan di memori peramban klien[9]. Pada model *flat canvas* tradisional, aplikasi mencoba memuat seluruh tabel, seluruh kolom, dan seluruh garis relasi *foreign key* dalam satu ruang kerja secara serentak, yang secara eksponensial meningkatkan kompleksitas perhitungan tata letak graf dan memicu kegagalan *rendering* ketika jumlah entitas melampaui ratusan tabel[10].

1. **Azimutt (Pembatasan Atribut Bertingkat dan Parsing SPA Client-Side):** Dirancang secara spesifik menggunakan arsitektur *Single-Page Application* (SPA berbasis Elm) untuk menangani basis data dunia nyata yang masif dan kompleks (*large and messy databases*), Azimutt mencegah kelebihan beban DOM melalui dua mekanisme penyaringan langsung di tingkat kanvas[9]. Pertama, Azimutt secara tegas menolak pendekatan yang menampilkan seluruh entitas sekaligus dalam satu diagram tunggal[13]. Kedua, mengingat satu entitas tabel perbankan dapat memiliki ratusan atribut yang membebani memori visual, Azimutt secara *default* membatasi *rendering* awal setiap entitas hanya pada **15 atribut terpenting**[13]. Pemilihan 15 atribut ini dihitung secara deterministik menggunakan urutan prioritas struktural: (1) *primary key*, (2) atribut yang memiliki relasi (*foreign keys*), (3) kolom yang memiliki *indexes*, dan terakhir (4) urutan deklarasi kolom di dalam SQL (*SQL order*)[13]. Selain itu, Azimutt mendukung pengabaian otomatis terhadap kolom-kolom audit standar yang bersifat repetitif di setiap

tabel—seperti created_by, updated_by, dan deleted_by beserta variasinya—sehingga kepadatan simpul DOM berkurang secara signifikan tanpa menghilangkan informasi relasional inti[17].

2. **DrawSQL (Pembatasan Kuota Entitas Kanvas dan Batas Ukuran Parser):** Untuk menjaga responsivitas kanvas interaktif di sisi peramban, DrawSQL menerapkan pendekatan pembatasan kuota tabel per diagram (*per-diagram table limit*) serta pembatasan ukuran *payload* pada tingkat parser DDL[11]. Pada lingkungan tanpa akun (/draw), kanvas dibatasi maksimal 20 tabel per diagram, di mana impor DDL yang melampaui kuota tersebut akan dipotong (*truncated*) secara otomatis pada 20 tabel pertama dan sisanya dicatat di dalam *import log*, sedangkan pada lingkungan tim berbayar batas kapasitas tabel ditingkatkan sesuai tingkatan paket[19]. Guna melindungi *engine* parser dari kelebihan beban memori, DrawSQL menetapkan batas ukuran dokumen maksimal **200KB untuk input SQL DDL** dan **500KB untuk objek *schema patch* JSON** pada antarmuka pemrogramannya[23]. Pernyataan SQL non-struktur seperti *views*, *triggers*, *stored procedures*, serta baris data INSERT secara otomatis dilewati (*skipped*) saat *rendering*, namun tetap dihitung terhadap batas kuota karakter parser; oleh sebab itu, ekstraksi *structure-only dump* menjadi syarat mutlak agar proses *rendering* diagram tidak gagal[11].
3. **Eraser.io (Arsitektur Hibrida Diagram-as-Code dan Kanvas Visual):** Eraser.io (dilengkapi *DiagramGPT*) mengadopsi paradigma hibrida yang menggabungkan sintaks *diagram-as-code* dengan antarmuka penyuntingan visual *point-and-click* (*live canvas*)[27]. Berbeda dari perangkat yang memaksakan pemuatan seluruh katalog skema sekaligus, *engine rendering* Eraser bekerja secara selektif mengikuti deklarasi tekstual yang ditulis oleh arsitek data[27]. Karena struktur visual diturunkan langsung dari definisi teks yang hanya mendeklarasikan simpul (*nodes*), pengelompokan (*groups*), dan koneksi (*connections*) yang relevan bagi suatu konteks arsitektur, kanvas hanya mengalokasikan elemen DOM untuk entitas yang didefinisikan di dalam blok kode tersebut, sehingga terhindar dari kelumpuhan performa akibat tabel-tabel periferal yang tidak relevan[27].

B. Mekanisme Manajemen Segmentasi Skema untuk Kebutuhan Multi-Tim

Di lingkungan perbankan, satu basis data produksi sering kali diakses oleh berbagai unit kerja dengan fokus domain yang sangat berbeda—sebagai contoh, tim *Credit Scoring* hanya membutuhkan visibilitas terhadap tabel profil risiko, agunan, dan riwayat pinjaman nasabah, sedangkan tim *Regulatory Reporting* berfokus pada tabel buku besar (*general ledger*), mutasi transaksi, dan rekapitulasi kepatuhan. Menampilkan ratusan tabel dari kedua domain tersebut dalam satu kanvas datar akan menghambat pemahaman struktural.

- **Folder-Based Partial Layouts pada Azimutt:** Azimutt menyelesaikan permasalahan segmentasi multi-tim melalui fitur arsitektural bernama **Layouts** (https://azimutt.app/docs/layouts)[13]. Alih-alih memelihara satu diagram raksasa, arsitek data dapat membangun banyak *layout* parsial di atas satu sumber skema yang sama, di mana setiap *layout* hanya menampilkan subset entitas dan atribut yang dipilih secara

spesifik untuk suatu kasus penggunaan atau cakupan tim (*team scope*)[10]. Untuk mengelola puluhan *layout* lintas divisi, Azimutt menyediakan sistem strukturisasi berbasis folder virtual yang diurutkan secara alfabetis menggunakan karakter garis miring (/) langsung pada penamaan *layout*—sebagai contoh, teams / scoring untuk kelompok model penilaian kredit, teams / reporting untuk pelaporan regulasi, atau teams / revenue / mrr untuk analitik pendapatan[13]. Keunggulan krusial dari pendekatan ini adalah kemampuan pelacakan lintas-tim: ketika seorang *engineer* membuka panel detail sebuah entitas tabel, Azimutt secara otomatis menampilkan daftar seluruh *layout* (misalnya baik teams / scoring maupun teams / reporting) yang memuat tabel tersebut, sehingga ketergantungan bersama (*shared table scope*) antar-tim langsung terdeteksi[13]. Lebih jauh lagi, di dalam setiap *layout*, tabel-tabel yang memiliki keterkaitan konseptual dapat dibungkus dengan latar belakang visual berwarna menggunakan fitur **Groups**, serta dilengkapi catatan aturan bisnis kontekstual menggunakan **Memos** berbasis Markdown yang ditempatkan langsung di atas kanvas[13].

- **Text-Based Subgraphs dan Batas Domain pada Eraser.io:** Berbeda dengan pendekatan *layout* berbasis antarmuka pada Azimutt, paradigma *diagram-as-code* pada **Eraser.io** (serta sintaks deklaratif sejenis seperti Mermaid) mengelola segmentasi melalui konstruksi **Subgraphs** atau pengelompokan tekstual di dalam kode sumber diagram[27]. Dalam pendekatan ini, batas-batas arsitektural—baik berupa batas kepemilikan tim (*team ownership boundary* seperti subgraf khusus *Credit Scoring* vs *Reporting*), batas jaringan (*network/trust boundary*), maupun batas penerapan (*deployment boundary*)—ditulis secara eksplisit sebagai blok kelompok di dalam sintaks teks[29]. *Engine* tata letak kemudian membaca aturan tata bahasa (*grammar rules*) tersebut untuk menggambar kotak pembatas (*subgraph boundary*) yang mengisolasi entitas-entitas milik satu tim dari tim lainnya beserta arah aliran relasinya[30]. Sementara itu, pada **DrawSQL**, segmentasi skema dilakukan melalui fitur pengelompokan visual berdasarkan domain (*group by domain*) di atas kanvas editor atau melalui pemisahan diagram ke dalam unit-unit kanvas spesifik di dalam ruang kerja (*workspace*) tim yang diakses menggunakan kontrol token per-tim[19].

C. Alur Otomatisasi Sinkronisasi Skema Skalabel (Stateful vs Stateless di Dalam Kontainer Docker)

Untuk menggantikan dokumentasi statis yang cepat usang tanpa melanggar prinsip *zero-credential-exposure*, proses pembaruan diagram ERD ketika terdapat penambahan tabel baru di basis data hulu harus diotomatisasi melalui mekanisme sinkronisasi berbasis teks dan *diff checking* di dalam *pipeline* internal bank[9]. Dalam merancang kontainer Docker pemodelan data lokal, terdapat dikotomi arsitektural antara pola operasi *stateless* dan *stateful*:

1. **Operasi Kontainer Stateless vs Stateful:** Dalam mode **stateless**, kontainer Docker hanya menjalankan peladen web ringan (seperti Nginx atau SPA *frontend*) yang merender berkas definisi skema statis (SQL DDL, DBML, atau JSON) secara langsung di memori peramban klien (in-memory atau localStorage) setiap kali halaman dimuat, tanpa menyimpan keadaan diagram di dalam basis data kontainer[7]. Sebaliknya, dalam pola

stateful berbasis Git (*Git-backed stateful workflow*), kondisi tata letak (*layouts*), posisi pengelompokan domain, serta metadata skema dipertahankan sebagai berkas terstruktur—seperti berkas proyek JSON Azimutt (*"just a structured JSON"*) atau berkas kode sumber diagram Eraser/DBML—yang disimpan dan diberi versi (*versioned*) di dalam repositori Git internal bank[12].

2. **Deteksi Perubahan Otomatis (*Diff Checking*) Tanpa Koneksi Database Live:** Ketika pengembang menambahkan tabel baru atau memodifikasi kolom pada basis data hulu, sinkronisasi ke platform visualisasi berlangsung secara deklaratif tanpa satu pun koneksi jaringan langsung dari kontainer ERD ke basis data produksi[10]:

- **Sinkronisasi Berbasis Git Diff dan Workspace Agent (Eraser.io & Diagram-as-Code):** Karena diagram direpresentasikan sebagai kode teks yang ramah terhadap Git (*Git-friendly syntax*), setiap perubahan skema yang dihasilkan oleh berkas migrasi dapat langsung dibandingkan menggunakan mekanisme *diff* standar pada *Pull Request* (PR)[27]. Melalui integrasi GitHub sync, ekstensi VS Code, atau agen CLI/MCP yang beroperasi di *workspace* repositori yang sama dengan kode aplikasi, sistem membaca perubahan struktur tabel di repositori, memutakhirkan definisi simpul entitas dan panah relasi di dalam berkas diagram, serta menyajikan perubahannya sebagai *diff* tekstual untuk ditinjau sebelum digabungkan (*merged*)[27].
- **Sinkronisasi Multi-Source JSON Schema (Azimutt):** Di dalam arsitektur Azimutt, seluruh format sumber (baik SQL DDL, Prisma, maupun AML) terlebih dahulu ditransformasikan menjadi **Azimutt JSON Schema** yang berfungsi sebagai sumber kebenaran tunggal (*ultimate truth*)[9]. Ketika skrip ekstraktor internal menghasilkan berkas *dump* SQL DDL atau JSON skema terbaru dari lingkungan hulu, berkas tersebut diinjeksikan kembali ke dalam proyek Azimutt[9]. Karena Azimutt memisahkan lapisan sumber skema (*sources*) dari lapisan tata letak visual (*layouts*), penambahan tabel baru pada berkas JSON/SQL akan langsung memperbarui katalog skema proyek dan dapat dimasukkan secara deklaratif ke dalam *layout* folder tim yang bersangkutan tanpa merusak konfigurasi *layout* yang telah ada[9].
- **Sinkronisasi REST API, Schema Patch, dan ETag (DrawSQL):** Untuk integrasi otomatis dari *pipeline* CI/CD, DrawSQL menyediakan endpoint REST API (POST https://api.drawsql.app/v1/diagrams) serta *Model Context Protocol* (MCP) server yang mengubah *payload* SQL DDL atau kontrak *schema patch* (GET /schema-patch-contract) menjadi diagram yang dapat disunting[22]. Untuk mendeteksi perubahan secara presisi (*diff checking*), endpoint GET /v1/diagrams/{uuid}/schema-full mendukung permintaan kondisional berbasis header **ETag** yang melacak setiap revisi skema[23]. Saat skema yang diperbarui diimpor ulang ke dalam diagram yang sudah berisi tabel, mekanisme impor DrawSQL bersifat aditif (tidak pernah menghapus kanvas yang ada) dan menyediakan opsi resolusi konflik **Override existing** (menimpa struktur tabel lama dengan definisi kolom baru) atau **Keep both tables**, sehingga tabel-tabel baru dari database hulu otomatis terintegrasi ke dalam kanvas domain[19].

Dimensi Evaluasi Arsitektural	Azimutt (https://azimutt.app/docs)	Eraser.io (https://eraser.io)	DrawSQL (https://drawsql.app/docs)
Strategi Efisiensi DOM & Penanganan Ratusan Tabel	Membatasi tampilan awal entitas hanya pada **15 atribut terpenting** (*PK, FK, index, SQL order*) dan menyembunyikan kolom audit repetitif[13].	Merender secara selektif hanya simpul (*nodes*), grup, dan relasi yang dideklarasikan di dalam blok kode *diagram-as-code*[27].	Menerapkan kuota tabel per diagram (maks 20 tabel di /draw), melewatkan objek non-tabel, dan membatasi *payload* DDL maksimal 200KB[11].
Mekanisme Segmentasi Skema Multi-Tim	**Folder-based Partial Layouts** dengan pemisah garis miring (teams / scoring, teams / reporting), **Groups** berwarna, dan **Memos** Markdown[13].	**Text-based Subgraphs / Groups** yang mendefinisikan batas domain (*ownership/trust boundary*) secara deklaratif di dalam kode[27].	Pengelompokan visual berbasis domain (*group by domain*) di atas kanvas dan isolasi diagram per token tim[20].
Arsitektur Penyimpanan State (Stateful vs Stateless)	Mendukung mode *stateless* (in-memory SPA) maupun *stateful* melalui ekspor/impor berkas **Azimutt Structured JSON** di repositori Git[9].	Hibrida *stateful* berbasis kode teks (*Git-diffable*) yang disinkronkan dua arah dengan repositori GitHub dan kanvas visual[27].	Penyimpanan draf lokal di localStorage peramban (/draw) atau persistensi *cloud* yang diakses melalui REST API (/schema-full)[21].
Mekanisme Otomatisasi Sync & Diff Checking (Zero-Credential)	Konversi multi-source (DDL/Prisma) ke *Azimutt JSON Schema* tanpa menghapus konfigurasi *Layouts*	Sinkronisasi GitHub PR *diff* dan pembaruan kode diagram otomatis melalui ekstensi VS Code atau agen *workspace*[27].	Injeksi DDL atau *schema patch* via REST API / MCP Server dengan pelacakan revisi berbasis **ETag** dan mode *Override*

	yang sudah ada[9].		*existing*[19].

3. Verifikasi Kemampuan Self-Hosting ChartDB dengan Docker dan Spesifikasi Teknisnya

A. Kapabilitas Self-Hosting ChartDB Menggunakan Docker dan Tautan Dokumentasi Resmi

Berdasarkan pemeriksaan terhadap repositori kode sumber resmi, **ChartDB benar-benar dapat di-*self-host* secara penuh menggunakan Docker di infrastruktur lokal atau internal tanpa bergantung pada layanan *cloud*/SaaS mereka (chartdb.io)**[7]. ChartDB merupakan perangkat lunak *open-source* berlisensi **GNU Affero General Public License v3.0 (AGPL-3.0)** yang dibangun menggunakan **TypeScript (99,8%)**[8]. Tim pengembang ChartDB mempublikasikan *official container image* melalui GitHub Container Registry (ghcr.io/chartdb/chartdb:latest) dan menyertakan berkas Dockerfile resmi di repositori utama mereka untuk kompilasi mandiri.

Dokumentasi resmi terkait *deployment* Docker ChartDB dapat diverifikasi langsung pada tautan-tautan berikut: halaman utama repositori dan instruksi Docker resmi di https://github.com/chartdb/chartdb, kode sumber Dockerfile resmi di https://github.com/chartdb/chartdb/blob/main/Dockerfile[34], catatan rilis resmi (termasuk pembaruan dan perbaikan *docker image* seperti pada rilis v1.18.1 hingga v1.20.1) di https://github.com/chartdb/chartdb/releases[6], rujukan resmi perbedaan versi Cloud vs Self-Hosted pada diskusi isu GitHub di https://github.com/chartdb/chartdb/issues/609 (yang menautkan dokumentasi https://docs.chartdb.io/docs/cloud-vs-self-hosted)[36], serta panduan implementasi kontainer di https://converter.brightcoding.dev/blog/chartdb-the-revolutionary-db-visualizer-every-developer-needs[7] dan https://blog.logrocket.com/mastering-charts-database-visualization-chartdb/[5].

Sebagai catatan kepatuhan keamanan yang sangat krusial bagi perbankan, tim ChartDB juga memiliki repositori GitHub Action terpisah di https://github.com/chartdb/chartdb-action (https://github.com/marketplace/actions/chartdb-database-sync)[18]. Namun, GitHub Action tersebut bekerja dengan meminta penyimpanan kredensial DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD, dan CHARTDB_API_TOKEN di dalam GitHub Secrets untuk melakukan koneksi langsung ke basis data. Sesuai dengan aturan *zero-credential-exposure* pada riset ini, **chartdb-action tidak boleh digunakan sama sekali**; bank wajib menggunakan kontainer mandiri ghcr.io/chartdb/chartdb:latest dengan metode penempelan hasil kueri JSON (*Smart Query*) secara statis[7].

B. Prosedur Deployment Docker dan Konfigurasi Hardening untuk Perbankan

Berdasarkan dokumentasi resmi di https://github.com/chartdb/chartdb, https://github.com/chartdb/chartdb/blob/main/Dockerfile, dan panduan produksi untuk lingkungan teregulasi di

https://converter.brightcoding.dev/blog/chartdb-the-revolutionary-db-visualizer-every-developer-needs, terdapat tiga skenario *deployment* Docker yang dapat diterapkan oleh tim IT bank. Skenario pertama adalah menjalankan *official image* secara langsung dalam mode privasi penuh (tanpa fitur AI eksternal dan tanpa analitik) dengan mengeksekusi perintah docker pull ghcr.io/chartdb/chartdb:latest diikuti oleh docker run -e DISABLE_ANALYTICS=true -p 8080:80 ghcr.io/chartdb/chartdb:latest, yang kemudian dapat diakses pada http://localhost:8080.
Skenario kedua adalah melakukan kompilasi lokal (*local build*) dengan menyembunyikan seluruh atribut layanan *cloud* ChartDB. Bedah kode sumber https://github.com/chartdb/chartdb/blob/main/Dockerfile mengungkap bahwa selama tahap kompilasi, Dockerfile menerima lima argumen (ARG) yang ditulis ke dalam berkas .env sebelum perintah npm run build dijalankan, yaitu VITE_OPENAI_API_KEY, VITE_OPENAI_API_ENDPOINT, VITE_LLM_MODEL_NAME, VITE_HIDE_CHARTDB_CLOUD, dan VITE_DISABLE_ANALYTICS. Di lingkungan perbankan, mengompilasi *image* dengan menyetel --build-arg VITE_HIDE_CHARTDB_CLOUD=true dan --build-arg VITE_DISABLE_ANALYTICS=true memastikan bahwa aplikasi web yang disajikan di jaringan internal bank tidak menampilkan tautan ke chartdb.io dan tidak memuat skrip telemetri eksternal[7].
Skenario ketiga ditujukan apabila bank tetap membutuhkan fitur *AI-Powered Export* (untuk menghasilkan skrip migrasi DDL lintas dialek, seperti dari MySQL ke PostgreSQL atau dari SQLite ke MariaDB) tanpa mengizinkan pengiriman metadata skema ke peladen OpenAI publik. Dokumentasi resmi (https://github.com/chartdb/chartdb) menyediakan konfigurasi **Custom Inference Server** untuk menghubungkan kontainer ChartDB ke peladen LLM lokal di dalam intranet bank (misalnya peladen **vLLM** internal pada http://localhost:8000/v1 yang menjalankan model Qwen/Qwen2.5-32B-Instruct-AWQ). Kompilasi dilakukan dengan perintah docker build --build-arg VITE_OPENAI_API_ENDPOINT=http://localhost:8000/v1 --build-arg VITE_LLM_MODEL_NAME=Qwen/Qwen2.5-32B-Instruct-AWQ -t chartdb . dan dijalankan dengan docker run -e OPENAI_API_ENDPOINT=http://localhost:8000/v1 -e LLM_MODEL_NAME=Qwen/Qwen2.5-32B-Instruct-AWQ -p 8080:80 chartdb. Dokumentasi resmi memperingatkan secara eksplisit agar administrator hanya memilih salah satu antara Opsi 1 (OpenAI API key) atau Opsi 2 (Custom endpoint dan nama model), dan dilarang mencampurkan kedua opsi tersebut.

C. Spesifikasi Teknis (RAM, Storage, dan Bedah Arsitektur Kontainer)

Terkait persyaratan perangkat keras spesifik, perlu dinyatakan secara terus terang bahwa di dalam dokumentasi resmi ChartDB (https://github.com/chartdb/chartdb dan https://github.com/chartdb/chartdb/blob/main/Dockerfile), **tidak ketemu** pencantuman angka batas minimum RAM (dalam MB/GB) maupun kapasitas *storage* disk secara eksplisit.
Namun, ketiadaan pencantuman angka RAM dan *storage* yang besar tersebut dapat dijelaskan secara pasti melalui bedah teknis terhadap berkas Dockerfile resmi ChartDB di https://github.com/chartdb/chartdb/blob/main/Dockerfile (yang berukuran sangat ringkas, yakni 34 baris kode atau 894 Bytes). Berkas Dockerfile tersebut menerapkan arsitektur *multi-stage build* dua tahap. Pada tahap pertama (FROM node:24-alpine AS builder, atau Node.js 18+ dan npm untuk pengembangan lokal), kontainer menyalin package.json dan package-lock.json ke direktori kerja /usr/src/app, menjalankan npm ci untuk memasang dependensi (React,

TypeScript, dan pustaka diagram react-flow/xyflow), membentuk berkas .env, lalu mengeksekusi npm run build untuk menghasilkan bundel aset statis di direktori /usr/src/app/dist.
Pada tahap kedua (FROM nginx:stable-alpine AS production), seluruh lingkungan Node.js dibuang dan kontainer produksi hanya menyalin hasil kompilasi statis dari /usr/src/app/dist ke direktori /usr/share/nginx/html milik peladen web **Nginx di atas Alpine Linux**, menyalin default.conf.template ke /etc/nginx/conf.d/default.conf.template, menyalin skrip /entrypoint.sh, serta membuka port EXPOSE 80. Karena kontainer *runtime* murni hanya menjalankan nginx:stable-alpine yang menyajikan berkas statis HTML, CSS, dan JavaScript tanpa proses *backend* aplikasi dan tanpa basis data internal, konsumsi memori (RAM) dan ukuran *storage* kontainer saat berjalan sangatlah kecil (setara dengan kontainer Nginx Alpine statis standar). Seluruh pemrosesan metadata skema JSON dan penggambaran kanvas diagram dilakukan 100% di sisi klien (*entirely client-side* di dalam memori peramban web pengguna), dan penyimpanan atau pembagian diagram dilakukan dengan mengekspor/mengimpor berkas JSON, gambar SVG/PNG, atau skrip SQL secara lokal[5]. Artinya, kontainer Docker ChartDB bersifat **100% *stateless*** dan tidak membutuhkan *persistent storage volume* untuk menyimpan data skema produksi.

Tahap / Komponen Kontainer ChartDB	**Spesifikasi Eksak Berdasarkan Dockerfile (https://github.com/chartdb/chartdb/blob/main/Dockerfile)**	**Karakteristik Penggunaan Sumber Daya (RAM, CPU, Storage)**
Tahap Kompilasi (builder)	Base image node:24-alpine (atau Node.js 18+ lokal); mengeksekusi npm ci dan npm run build di /usr/src/app.	Membutuhkan alokasi RAM dan CPU sementara hanya pada saat proses kompilasi aset TypeScript/React berlangsung.
Tahap Produksi (production)	Base image nginx:stable-alpine; menyajikan aset statis dari /usr/share/nginx/html pada port 80 melalui /entrypoint.sh.	Sangat ringan; tidak ada *runtime* Node.js atau basis data di sisi server saat kontainer berjalan.
Penyimpanan & Pemrosesan Skema	Pemrosesan JSON skema dilakukan sepenuhnya di sisi klien (*client-side* di peramban); ekspor/impor via berkas JSON/SQL/SVG.	Kebutuhan *persistent disk storage* di server Docker adalah **0 MB** (*stateless*); beban *rendering* ratusan tabel ditanggung RAM

		peramban klien.
Dukungan Dialek Basis Data	PostgreSQL, MySQL, SQL Server, MariaDB, SQLite (+ Cloudflare D1), CockroachDB, ClickHouse, serta impor/ekspor DBML dan CSV metadata[6].	Tidak memerlukan instalasi *database driver* di dalam kontainer karena ekstraksi dilakukan via *Smart Query* di klien DB masing-masing.

4. Evaluasi Mendalam dbt docs generate pada Transformasi BigQuery: Zero-Credential-Exposure atau Butuh Service Account?

Apabila proses transformasi dari lapisan *Data Raw* ke *Data Mart* di BigQuery dibangun menggunakan **dbt (data build tool)**, apakah eksekusi perintah dbt docs generate termasuk dalam kategori *zero-credential-exposure*, atau tetap membutuhkan *Service Account* BigQuery?

Jawaban teknis yang eksak adalah: **Perintah dbt docs generate TIDAK termasuk *zero-credential-exposure* pada saat perintah tersebut dieksekusi, karena ia TETAP MEMBUTUHKAN koneksi aktif (*Service Account* BigQuery) ke *data warehouse* untuk menarik metadata fisik tabel guna membentuk berkas catalog.json**[37]. Meskipun demikian, karena keluaran akhir dari dbt docs generate hanyalah sekumpulan berkas situs web statis (index.html beserta artefak JSON/Parquet), **hosting atau penyajian portal dokumentasi tersebut di dalam kontainer Docker dapat dioperasikan secara 100% *zero-credential-exposure* setelah artefak statisnya terbentuk**[37].

Bukti dokumentasi resmi dan referensi teknis mengenai perilaku ini dapat diperiksa pada dokumentasi resmi perintah dbt docs (mencakup dbt v1 dan dbt v2) di https://docs.getdbt.com/reference/commands/cmd-docs[40], dokumentasi resmi perintah dbt compile dan dbt parse di https://docs.getdbt.com/reference/commands/compile[42], dokumentasi artefak dbt pada integrasi katalog DataHub di https://docs.datahub.com/docs/generated/ingestion/sources/dbt[43], panduan penggunaan dbt parse tanpa koneksi basis data di https://godatadriven.github.io/dbt-bouncer/v3.8.0/getting_started/[44], analisis mekanisme koneksi dbt docs generate di https://npblue.com/data/dbt/dbt-docs-generate/[38], https://phillIant.com/posts/20260330-dbt-docs/[37], dan https://popsql.com/learn-dbt/dbt-docs[24], serta arsitektur produksi *self-hosted* dbt-docs-hub di https://medium.com/towardsdev/deploying-dbt-docs-in-production-for-multiple-projects-self-hosted-and-secured-part-1-3-64aa43229329[41].

Level 1: Bedah Dua Artefak Inti (manifest.json vs catalog.json)

Ketika dbt docs generate dijalankan pada proyek dbt, sistem memproduksi dua artefak metadata utama di dalam direktori target/ yang memiliki perbedaan fundamental dalam hal

kebutuhan kredensial[37]. Artefak pertama adalah **manifest.json**, yang memuat graf proyek secara menyeluruh—mencakup setiap *model*, *source*, *seed*, *snapshot*, *test*, *exposure*, *semantic model*, *metric*, dan *macro* beserta deskripsi yang ditulis di dalam berkas .yml maupun blok dokumentasi Markdown ({% docs ... %} pada berkas .md). Seluruh struktur silsilah data (*data lineage graph* yang memetakan hubungan hulu ke hilir antara tabel *Raw* {{ source() }} dan tabel *Data Mart* {{ ref() }}) dihitung murni dari kode sumber proyek dan disimpan di dalam manifest.json[37].

Artefak kedua adalah **catalog.json**, yang menyimpan metadata level skema fisik yang ditarik langsung dari *data warehouse* BigQuery untuk tabel dan *views* yang dihasilkan oleh model di dalam proyek[37]. Berkas catalog.json memuat nama kolom aktual di gudang data, tipe data kolom fisik (*warehouse column types*), serta statistik tabel seperti jumlah baris (*row counts*) dan ukuran tabel[37]. Untuk menghasilkan catalog.json, dbt docs generate **wajib membuka koneksi aktif menggunakan kredensial (*Service Account* BigQuery)** guna menjalankan kueri introspeksi metadata terhadap BigQuery[10]. Apabila dbt docs generate dijalankan tanpa didahului oleh pembangunan model di gudang data, catalog.json hanya akan mencerminkan tabel apa pun yang kebetulan sudah ada di *warehouse* pada saat itu; dan apabila proses pembuatan catalog.json terputus atau gagal karena ketiadaan koneksi/izin, antarmuka dbt docs serve pada dbt v1 akan gagal dijalankan[46].

Level 2: Perintah Kompilasi Murni Tanpa Koneksi Database (dbt parse dan Flag --no-introspect)

Apabila tim IT bank ingin mengekstrak struktur *data lineage* (manifest.json) di dalam sebuah kontainer CI/CD yang sama sekali tidak diizinkan memiliki *Service Account* BigQuery, dokumentasi resmi dbt (https://docs.getdbt.com/reference/commands/compile dan https://godatadriven.github.io/dbt-bouncer/v3.8.0/getting_started/[47]) menyediakan perintah khusus **dbt parse**. Dokumentasi resmi dbt menyatakan secara eksplisit bahwa jika pengguna hanya ingin dbt membaca dan memvalidasi kode proyek tanpa terhubung ke *data warehouse*, pengguna harus menggunakan dbt parse. Perintah dbt parse menghasilkan artefak manifest.json (yang sudah mencakup seluruh graf *lineage* model dan sumber) dengan **nol koneksi basis data (*no database connection required*)**[42]. Selain itu, pada perintah dbt compile, administrator dapat menyertakan flag global --no-populate-cache (untuk menonaktifkan pengisian *cache* metadata awal) dan flag --no-introspect (contoh: dbt --no-populate-cache compile --no-introspect) guna memblokir eksekusi kueri introspektif ke BigQuery.

Level 3: Perubahan Arsitektur pada dbt v2 (dbt Docs v2 dan Flag --write-catalog)

Berdasarkan dokumentasi resmi terbaru di https://docs.getdbt.com/reference/commands/cmd-docs, terdapat evolusi arsitektur yang signifikan pada **dbt v2.0 ke atas (dbt Docs v2)** untuk alur kerja lokal dan *self-hosted*. Alih-alih memuat berkas manifest.json statis berukuran besar langsung di peramban web, dbt v2 memproduksi artefak berformat **Parquet** ketika proyek dikompilasi atau dibangun. Perintah dbt docs generate pada v2 mengompilasi proyek, menulis indeks di bawah target/index/, dan

mengekspor situs dokumentasi statis ke direktori target/ (target/index.html dan target/assets/). Pada arsitektur dbt v2 ini, situs dbt Docs v2 membaca metadata dari **dbt Information Schema** (target/info_schema/v1/), bukan lagi dari berkas catalog.json (di mana catalog.json kini tetap disediakan sebagai artefak terpisah untuk kebutuhan alat eksternal seperti DataHub atau Wren AI)[40]. Untuk menghasilkan catalog.json beserta tipe kolom fisik gudang data dan mengaktifkan *column-level lineage* pada dbt v2, pengembang menggunakan flag --write-catalog yang dikombinasikan dengan --generate-info-schema pada perintah dbt build, dbt run, dbt parse, atau dbt compile (misalnya dbt build --write-catalog atau dbt compile --generate-info-schema --static-analysis strict)[40].

Level 4: Pola Deployment Docker Zero-Credential untuk Portal Dokumentasi dbt di Bank

Karena hasil akhir dari dbt docs generate merupakan situs web statis murni (index.html beserta artefak pendamping di dalam folder target/), dokumentasi resmi dbt (https://docs.getdbt.com/reference/commands/cmd-docs) dan panduan arsitektur produksi (https://medium.com/towardsdev/deploying-dbt-docs-in-production-for-multiple-projects-self-hosted-and-secured-part-1-3-64aa43229329 serta https://phillIant.com/posts/20260330-dbt-docs/) menunjukkan bahwa kontainer dokumentasi tidak boleh menjalankan koneksi ke BigQuery. Sebaliknya, *Orchestrator* internal (yang menjalankan pipeline transformasi harian di lingkungan aman) memicu dbt docs generate pada akhir eksekusi *pipeline*, lalu mengunggah berkas-berkas statis dari direktori target/ (dbt docs generate && mv target public) ke penyimpanan artefak internal (*Artifact Store*). Selanjutnya, kontainer Docker dokumentasi (seperti arsitektur **dbt-docs-hub** yang terdiri dari 3 komponen: *sidecar* penyinkron artefak, *FastAPI* sebagai *Filtering API* yang berkonsultasi dengan *Policy Engine* untuk menyaring akses kolom PII, dan *Nginx* sebagai peladen web statis) hanya menyajikan berkas statis tersebut kepada pengguna. Dengan pola pemisahan ini, kontainer Docker dokumentasi yang diakses oleh analis bank beroperasi dengan **nol kredensial BigQuery (*zero-credential-exposure*)**.

Perintah / Tahap Eksekusi dbt	Tautan Dokumentasi Resmi	Artefak yang Dihasilkan di target/	Butuh Service Account / Koneksi Live BigQuery?	Peran dalam Arsitektur Dokumentasi & Lineage Bank
dbt parse	https://docs.getdbt.com/reference/commands/compile	manifest.json (mencakup graf proyek & *lineage* logis antar-model)[47].	**TIDAK BUTUH** (*Zero database connection*)[47].	Validasi sintaks dan ekstraksi graf *lineage* murni dari kode di CI/CD tanpa kredensial[47].

dbt compile --no-introspect	https://docs.getdbt.com/reference/commands/compile	Berkas SQL terkompilasi & manifest.json (dikombinasikan dengan --no-populate-cache).	**TIDAK BUTUH** (Memblokir kueri introspektif ke *warehouse*).	Menghasilkan skrip SQL matang untuk dianalisis lebih lanjut oleh parser *offline* seperti ZetaSQL[3].
dbt docs generate (dan --write-catalog)	https://docs.getdbt.com/reference/commands/cmd-docs	index.html, manifest.json, catalog.json (v1) atau artefak Parquet & target/index/ (v2)[37].	**WAJIB BUTUH** (Menarik tipe kolom fisik & statistik tabel dari BigQuery)[37].	Dieksekusi satu kali oleh *orchestrator* internal di akhir *pipeline* untuk mencetak bundel statis[37].
Hosting Kontainer Web (Nginx / dbt-docs-hub)	https://docs.getdbt.com/reference/commands/cmd-docs	Menyajikan berkas statis dari folder target/ (mv target public) melalui HTTP.	**TIDAK BUTUH** (*100% Zero-Credential-Exposure*).	Portal dokumentasi *self-hosted* di Docker yang diakses oleh tim bank tanpa kredensial DB.

5. Studi Kasus Implementasi Dokumentasi Skema Otomatis pada Sektor Finansial yang Teregulasi Ketat

Dalam lanskap rekayasa platform (*Platform Engineering*) di institusi perbankan digital dan perusahaan teknologi finansial (*fintech*) yang beroperasi di bawah pengawasan regulasi ketat, pengelolaan dokumentasi skema lintas platform telah bergeser dari aktivitas menggambar manual menuju pendekatan rekayasa perangkat lunak berbasis metadata terstruktur (*structured metadata*) dan *Diagram-as-Code*[27]. Studi kasus rekayasa platform berikut membedah anatomi transisi arsitektural dari dokumentasi statis yang terfragmentasi menuju *pipeline* ekstraksi skema otomatis yang berjalan sepenuhnya di dalam perimeter internal bank tanpa eksposur kredensial produksi (*Zero-Production-Touch* / *Zero-Credential-Exposure*)[7].

A. Tantangan Dokumentasi Statis Terfragmentasi dan Akumulasi Utang Teknis (Technical Debt)

Pada fase pertumbuhan awal hingga menengah, banyak institusi finansial memelihara dokumentasi *Entity-Relationship Diagram* (ERD) secara manual menggunakan kanvas penggambaran bebas seperti **Draw.io** (diagrams.net), Lucidchart, atau halaman wiki internal

yang dibagi ke dalam puluhan tab terpisah untuk masing-masing modul layanan[27]. Seiring bertambahnya skala sistem perbankan hingga mencakup ratusan tabel yang tersebar di berbagai mesin basis data produksi (PostgreSQL untuk layanan mikro inti, MySQL/MariaDB untuk kanal transaksi, dan Microsoft SQL Server untuk sistem pelaporan warisan), pendekatan dokumentasi manual ini menciptakan utang teknis (*technical debt*) yang akut pada tiga titik kritis[7]:

1. **Keusangan Instan (*Schema Drift*) dan Risiko Operasional:** Halaman wiki atau berkas diagram statis langsung mengalami keusangan (*stale*) tepat pada detik seorang *engineer* menambahkan kolom baru, mengubah tipe data, atau mengganti nama tabel melalui skrip migrasi basis data tanpa memperbarui gambar ERD secara manual[29]. Dalam praktik rekayasa perangkat lunak, untuk memperbarui satu relasi pada diagram manual, *engineer* harus membuka berkas kanvas, menggeser kotak dan menarik ulang garis konektor dengan tetikus, mengekspor ulang berkas gambar (PNG/SVG), lalu mengunggah *blob* biner tersebut ke portal dokumentasi[29]. Akibat tingginya friksi kerja ini, dokumentasi visual tertinggal jauh dari realitas skema produksi, padahal diagram arsitektur yang usang lebih berbahaya daripada ketiadaan diagram sama sekali karena memberikan rujukan yang salah saat tim *on-call* menangani insiden produksi atau saat peninjau keamanan (*security reviewer*) mengaudit aliran data nasabah[10].
2. **Ketiadaan Standarisasi Berbasis Teks dan Rendahnya Diffability pada Pull Request:** Berkas diagram visual konvensional disimpan dalam format kanvas proprietari atau struktur XML grafis yang rumit, sehingga tidak dapat ditinjau perubahannya secara baris-demi-baris (*diffable*) oleh rekan sejawat di dalam *Pull Request* (PR) Git bersama-sama dengan perubahan kode aplikasi[27]. Tanpa tata bahasa deklaratif yang ketat (*strict grammar rules*), setiap *engineer* menggambar simbol entitas, kardinalitas, dan tipe data dengan konvensi visual yang berbeda-beda di puluhan tab Draw.io[30].
3. **Kegagalan Menangani Dependensi Multi-Platform dan Basis Data Warisan:** Ketika tim mewarisi basis data berusia belasan tahun dengan lebih dari 200 tabel tanpa definisi *foreign key* eksplisit dan tanpa dokumentasi yang akurat, kanvas manual tidak memiliki kemampuan analitik untuk menginferensi relasi antar-tabel, mendeteksi duplikasi struktur, ataupun menyatukan skema dari berbagai dialek SQL ke dalam satu standar referensi tunggal[7].

B. Arsitektur Pipeline Ekstraksi Skema Teks Offline (Kontainer Docker "Schema Extractor")

Untuk mengeliminasi pemeliharaan manual sekaligus mematuhi larangan keras penyimpanan kredensial basis data produksi pada aplikasi visualisasi (*zero-credential-exposure*), tim *Platform Engineering* mengimplementasikan arsitektur dua lapis yang memisahkan **kontainer ekstraktor skema internal (*Schema Extractor*)** dari **kontainer penyaji visualisasi (*Diagram-as-Code Renderer*)**[7]. Pola ini mengadopsi praktik terbaik rekayasa perangkat lunak untuk analitik data berskala masif—sebagaimana diterapkan oleh institusi perbankan digital seperti **Nubank** (https://building.nu.com/scaling-data-analytics-with-software-engineering-best-practices/)

yang mewajibkan ekstraksi metadata terstruktur (*structured metadata*) secara terprogram di dalam ekosistem internal mereka (https://github.com/nubank/NuOpenMetadata)[49].
Di dalam arsitektur ini, kontainer Docker **Schema Extractor** dijalankan sebagai *scheduled job* terisolasi di dalam jaringan internal atau dipicu secara otomatis oleh *runner* CI/CD internal setiap kali terjadi perubahan pada direktori migrations/** atau schema/** di repositori utama[10]. Kontainer ini beroperasi melalui tiga tahapan mekanis tanpa pernah menyentuh baris data tabel produksi (*schema structure only, never row data*)[10]:

1. **Ekstraksi Offline Structure-Only Dump Multi-Platform:** Kontainer *Schema Extractor* mengeksekusi utilitas ekstraksi struktur murni bawaan masing-masing DBMS menggunakan akun *read-only* internal yang hanya diizinkan membaca katalog struktur, atau mengekstrak DDL langsung dari berkas migrasi/ORM di repositori kode tanpa menyentuh peladen basis data sama sekali[7]:
   - Pada mesin **PostgreSQL**, kontainer mengeksekusi perintah pg_dump --schema-only --no-owner --no-privileges DATABASE > schema.sql untuk menghasilkan instruksi DDL murni tanpa metadata kepemilikan pengguna dan tanpa data transaksi[11].
   - Pada mesin **MySQL / MariaDB**, kontainer mengeksekusi perintah mysqldump --no-data --skip-comments -u USERNAME -p DATABASE > schema.sql[11].
   - Pada mesin **Microsoft SQL Server**, ekstraksi menghasilkan skrip *Schema only* yang setiap pernyataannya dipisahkan secara ketat menggunakan tanda titik koma (*semicolon*)[11].
   - Pada layanan mikro yang skemanya dikelola sepenuhnya oleh ORM atau berkas migrasi di Git, kontainer mengekstrak DDL secara *offline* langsung dari kode sumber menggunakan perintah npx prisma migrate diff --from-empty --to-schema-datamodel prisma/schema.prisma --script (untuk Prisma), php artisan schema:dump (untuk Laravel), atau membaca langsung berkas db/structure.sql dan db/schema.rb[10].
2. **Transpilasi Otomatis ke Manifest Diagram-as-Code (DBML, Azimutt JSON, atau Eraser/Mermaid):** Berkas schema.sql statis yang dihasilkan pada tahap pertama kemudian diproses secara lokal di dalam kontainer ekstraktor untuk dikonversi menjadi kode manifest *Diagram-as-Code* terstandar—seperti berkas **DBML**, berkas **Azimutt JSON Schema**, atau sintaks deklaratif **Eraser / Mermaid erDiagram**—lengkap dengan pemetaan *primary keys*, *unique constraints*, *indexes*, nilai *enum*, serta *foreign keys*[6]. Selama tahap konversi ini, skrip otomasi juga menerapkan aturan segmentasi domain (seperti pengelompokan ke dalam *Folder-based Layouts* teams/scoring dan teams/reporting pada struktur JSON Azimutt atau pembungkusan entitas ke dalam blok subgraph pada sintaks *diagram-as-code*)[13].
3. **Distribusi Satu Arah ke Kontainer Visualisasi Client-Side (Zero-Credential-Exposure):** Artefak manifest statis yang telah tervalidasi tersebut selanjutnya di-commit ke repositori dokumentasi atau dikirim satu arah ke penyimpanan artefak statis untuk disajikan oleh kontainer Docker visualisasi internal (seperti kontainer **Azimutt** atau **ChartDB** berbasis nginx:stable-alpine yang dikompilasi dengan argumen VITE_HIDE_CHARTDB_CLOUD=true dan VITE_DISABLE_ANALYTICS=true)[7]. Karena seluruh proses *parsing* dan penggambaran diagram ERD dari berkas SQL/JSON dilakukan

sepenuhnya di sisi klien (*entirely client-side* di dalam peramban web pengguna), kontainer visualisasi ERD sama sekali tidak menyimpan kredensial, tidak memiliki koneksi jaringan ke basis data produksi, dan menjamin kepatuhan mutlak terhadap prinsip *Zero-Production-Touch*[7].

C. Dampak Kuantitatif terhadap Efisiensi Kerja Tim Rekayasa Data dan Perbankan

Transisi dari pemeliharaan ERD manual di Draw.io menuju arsitektur *Automated Diagram-as-Code* yang tersegmen memberikan dampak kuantitatif dan operasional yang terukur pada skala *enterprise* perbankan:

1. **Eliminasi 100% Diagram ERD Manual yang Usang (*Broken/Stale Visual Assets*):** Dengan menjadikan berkas migrasi SQL dan *structure-only dump* sebagai satu-satunya sumber kebenaran (*single source of truth*) yang meregenerasi kode manifest diagram secara otomatis setiap kali terjadi penggabungan kode (*merge*) ke cabang utama (main), seluruh aset visual yang usang atau terputus (*schema drift*) berhasil dieliminasi hingga **100%**[10]. Kehadiran fitur pelacakan waktu publikasi (*published timestamps* dan *reruns*) memberikan kepastian instan kepada tim audit dan *engineer* bahwa diagram yang mereka lihat mencerminkan struktur skema produksi hari ini, bukan kondisi beberapa bulan lalu[10].
2. **Pemangkasan Waktu Onboarding Engineer Baru dari Hitungan Minggu Menjadi Hari:** Berdasarkan tolok ukur implementasi nyata pada dokumentasi basis data warisan (*legacy database*) berskala **lebih dari 200 tabel** tanpa dokumentasi sebelumnya, penyediaan peta visual interaktif yang dikombinasikan dengan segmentasi *Layouts* per domain (memisahkan fokus skema teams/scoring dari teams/reporting serta membatasi tampilan awal pada 15 atribut kunci) terbukti memangkas waktu *onboarding engineer* baru untuk memahami model data secara drastis dari **beberapa minggu menjadi hanya beberapa hari** (*onboarding time drops from weeks to days*)[7]. Selain itu, penyederhanaan ekstraksi berbasis teks statis menghemat waktu konfigurasi dan penyiapan (*setup time*) selama **berjam-jam** dibandingkan konfigurasi koneksi basis data tradisional[7].
3. **Deteksi Otomatis Tabel Yatim Piatu (*Orphan Tables*) dan Anomali Skema di Pipeline CI/CD Sebelum Deployment:** Karena skema telah berbentuk kode teks terstruktur (*Diagram-as-Code* / JSON Schema), proses validasi kualitas pemodelan data dapat digeser ke kiri (*shift-left*) langsung ke dalam *pipeline* CI/CD pada tahap peninjauan *Pull Request*[10]. Mesin *database linter* (seperti kapabilitas *linter* bawaan Azimutt, validasi parser DDL/DBML pada ChartDB dan DrawSQL, maupun mesin validasi arsitektur otomatis) secara otomatis memindai manifest skema sebelum masuk ke tahap *deployment* untuk mendeteksi: (a) **tabel yatim piatu (*orphan tables*)** atau tabel yang kehilangan relasi *foreign key* akibat penghapusan kolom di tabel hulu, (b) ketidaksesuaian tipe data antar-kunci relasi lintas skema, (c) ketiadaan batasan NOT NULL pada kolom *primary key*, serta (d) pelanggaran aturan arsitektur (seperti peringatan otomatis WARN-01 hingga WARN-05 dan kegagalan parser akibat kesalahan sintaks DDL)[6]. Seluruh anomali tersebut tercatat secara eksplisit di dalam *import/validation log* pada *Pull*

Request, sehingga mencegah masuknya cacat desain skema ke lingkungan produksi perbankan[11].

Indikator Kinerja Utama (KPI) Platform EDM	Kondisi Sebelum Transisi (Dokumentasi Manual Draw.io)	Kondisi Sesudah Transisi (Automated Diagram-as-Code & Segmented Layouts)	Bukti Mekanisme & Referensi Arsitektural
Akurasi Sinkronisasi & Aset Visual Usang (*Stale Diagrams*)	Rentan mengalami *schema drift* sesaat setelah penambahan kolom/tabel; membutuhkan gambar ulang dan ekspor PNG manual[29].	**Eliminasi 100% diagram manual usang**; diagram diregenerasi otomatis dari DDL/migrasi setiap *merge* ke main dengan *freshness timestamps*[10].	Sinkronisasi *Git-diffable* dan regenerasi otomatis dari *source of truth*[10].
Waktu Onboarding Engineer Baru (Skala 200+ Tabel)	Membutuhkan waktu **beberapa minggu** (*weeks*) untuk menelusuri ratusan tabel tanpa dokumentasi dan tanpa *foreign keys*[7].	**Turun menjadi hitungan hari (*drops from weeks to days*)** dan menghemat berjam-jam waktu penyiapan (*setup time*)[7].	Segmentasi *Folder-based Layouts* (teams/scoring, teams/reporting) dan batas 15 atribut inti[7].
Deteksi Tabel Yatim Piatu (*Orphan Tables*) & Anomali Tipe Data	Tidak terdeteksi hingga menimbulkan *error* di lingkungan hilir atau ditemukan secara reaktif melalui inspeksi manual[7].	**Deteksi otomatis 100% di tahap CI/CD (PR Review)** sebelum *deployment* melalui *database linter* dan *parser log*[11].	*Database linter* Azimutt, penegakan NOT NULL PK & tipe DDL pada ChartDB/DrawSQL, serta validasi WARN-01–WARN-05[6].
**Eksposur Kredensial Produksi (*Credential*	Berisiko tinggi apabila menggunakan *plugin* penarik	**0% Eksposur Kredensial (*Zero-Production-Touch*)**;	Ekstraksi pg_dump --schema-only / mysqldump --no-data menuju

***Exposure*)**	skema yang menyimpan *connection string* produksi di aplikasi pihak ketiga[7].	pemrosesan menggunakan *structure-only dump* yang dirender 100% *client-side*[7].	kontainer statis nginx:stable-alpine[1][1].

Sintesis Rekomendasi Arsitektur untuk Ujian Komprehensif

Berdasarkan seluruh bukti dokumentasi resmi dan studi kasus perbankan di atas, rancangan arsitektur Enterprise Data Management (EDM) yang paling kokoh untuk diajukan pada ujian komprehensif adalah memisahkan secara tegas antara **perangkat dokumentasi ERD untuk lapisan sumber OLTP** dan **perangkat pelacakan Data Lineage untuk lapisan transformasi OLAP BigQuery**, di mana keduanya dijalankan di atas kontainer Docker internal tanpa menyimpan satu pun kredensial produksi (*Zero-Credential-Exposure*).

Pada lapisan sumber OLTP yang terdiri dari beragam mesin basis data produksi, tim IT bank dapat men-deploy kontainer Docker **ChartDB** (ghcr.io/chartdb/chartdb:latest yang berjalan di atas nginx:stable-alpine dengan konfigurasi DISABLE_ANALYTICS=true dan VITE_HIDE_CHARTDB_CLOUD=true sebagaimana tertera pada https://github.com/chartdb/chartdb/blob/main/Dockerfile) atau **Azimutt** (https://azimutt.app/docs/sources dalam mode *self-hosted* dengan fitur *Layouts* untuk mengelompokkan ratusan tabel menjadi tampilan domain yang terstruktur[9]). Administrator basis data hanya perlu menjalankan kueri metadata read-only satu kali (*Smart Query*) atau mengekspor *schema-only dump* (pg_dump --schema-only / mysqldump --no-data), lalu mengunggah berkas JSON atau SQL DDL statis tersebut ke antarmuka peramban ChartDB atau Azimutt yang memproses seluruh visualisasi secara *client-side* tanpa koneksi jaringan ke basis data OLTP[7].

Selanjutnya, pada lapisan transformasi *Data Raw* ke *Data Mart* di Google BigQuery, bank dapat mengombinasikan **dbt** dengan kapabilitas native **Dataplex Data Lineage API** (roles/datalineage.viewer di dalam perimeter VPC Service Controls[52]) atau mengadopsi arsitektur **Monzo Bank** (https://medium.com/data-monzo/mapping-our-data-journey-with-column-lineage-56209c00606d[5]) yang mem-parsing berkas SQL terkompilasi dari dbt menggunakan **ZetaSQL** secara *offline* dan menggabungkannya dengan metadata **BigQuery Audit Logs**[5]. Apabila diperlukan introspeksi terhadap INFORMATION_SCHEMA di BigQuery untuk mencetak catalog.json pada dbt docs generate, akses tersebut dijalankan murni oleh *orchestrator* internal menggunakan *Custom IAM Role* berbasis roles/bigquery.metadataViewer (--dataset_view=METADATA) yang diperketat dengan *IAM Conditions* (https://docs.cloud.google.com/bigquery/docs/conditions[47]). Berkas dokumentasi dan graf *lineage* statis yang dihasilkan (index.html, manifest.json, catalog.json, atau artefak Parquet pada dbt v2) kemudian disalin satu arah ke dalam kontainer Docker Nginx internal

(https://medium.com/towardsdev/deploying-dbt-docs-in-production-for-multiple-projects-self-hosted-and-secured-part-1-3-64aa43229329[40]), sehingga menghasilkan sistem dokumentasi skema dan *data lineage* otomatis yang memenuhi standar keamanan perbankan secara mutlak.

Works cited

1. An introduction to Monzo's data stack, https://monzo.com/blog/2021/10/14/an-introduction-to-monzos-data-stack
2. Petabyte Scale Data Management for Cloud Banking - Infosys, https://www.infosys.com/iki/perspectives/cloud-banking.html
3. Mapping our data journey with column lineage | by Borja Vazquez, https://medium.com/data-monzo/mapping-our-data-journey-with-column-lineage-56209c00606d
4. The many layers of data lineage - Medium, https://medium.com/data-monzo/the-many-layers-of-data-lineage-2eb898709ad3
5. Mastering charts and database visualization with ChartDB, https://blog.logrocket.com/mastering-charts-database-visualization-chartdb/
6. Releases · chartdb/chartdb - GitHub, https://github.com/chartdb/chartdb/releases
7. ChartDB: The DB Visualizer Every Developer Needs - Smart Converter, https://converter.brightcoding.dev/blog/chartdb-the-revolutionary-db-visualizer-every-developer-needs
8. GitHub - chartdb/chartdb: Database diagrams editor that allows you, https://github.com/chartdb/chartdb
9. Azimutt documentation > Sources, https://azimutt.app/docs/sources
10. Azimutt Alternative for Postgres Schemas - SchemaLens, https://schemalens.net/compare/azimutt-alternative
11. Import SQL (DDL) - DrawSQL, https://drawsql.app/docs/import-from-ddl
12. Azimutt documentation > Create your project, https://azimutt.app/docs/create-your-project
13. Azimutt documentation > Layouts, https://azimutt.app/docs/layouts
14. Document your database schema using rich text memos - Azimutt, https://azimutt.app/blog/document-your-database-with-memos
15. create-your-project.html.heex - GitHub, https://github.com/azimuttapp/azimutt/blob/main/backend/lib/azimutt_web/templates/website/docs/create-your-project.html.heex
16. azimutt/backend/lib/azimutt_web/templates/website/docs/badge.html, https://github.com/azimuttapp/azimutt/blob/main/backend/lib/azimutt_web/templates/website/docs/badge.html.heex
17. Azimutt - Explore your database, thanks to Elm ^^ - Show and Tell, https://discourse.elm-lang.org/t/azimutt-explore-your-database-thanks-to-elm/7734
18. ChartDB Database Sync · Actions · GitHub Marketplace, https://github.com/marketplace/actions/chartdb-database-sync
19. Import your schema - DrawSQL, https://drawsql.app/docs/import-your-schema

20. SQL to ERD — Paste SQL or DDL, Get a Database Diagram, https://drawsql.app/sql-to-erd
21. Overview - DrawSQL, https://drawsql.app/docs/draw-overview
22. DrawSQL for Developers | API and MCP Docs, https://drawsql.app/developers
23. DrawSQL API Reference — Interactive OpenAPI Explorer, https://drawsql.app/developers/api/reference
24. dbt docs generate & serve: Command Usage and Examples - PopSQL, https://popsql.com/learn-dbt/dbt-docs
25. Columns - DrawSQL, https://drawsql.app/docs/columns
26. chartdb-action - GitHub, https://github.com/chartdb/chartdb-action
27. Best AI Diagram Tools for Engineers and Claude Code Workflows, https://nimbalyst.com/blog/best-ai-diagram-tools-2026/
28. How to use AI to create better technical diagrams : r/dataengineering, https://www.reddit.com/r/dataengineering/comments/1jmmf52/how_to_use_ai_to_create_better_technical_diagrams/
29. Code to Diagram: Using AI to Generate Mermaid.js and Flowcharts, https://www.stackexpertise.com/ai-tools/code-to-diagram-using-ai-to-generate-mermaidjs-and-flowcharts
30. Turn Plain Text into Draw.io Diagrams, https://diagrams.so/generate/text-to-diagram
31. Ask HN: Guidelines for making clear architecture diagrams, https://news.ycombinator.com/item?id=40226897
32. GitHub - riyanamanda/helpdesk-backend: REST API for IT Helpdesk, https://github.com/riyanamanda/helpdesk-backend
33. How to create a software architecture diagram - Planloom, https://planloom.app/guides/create-a-software-architecture-diagram
34. chartdb/Dockerfile at main - GitHub, https://github.com/chartdb/chartdb/blob/main/Dockerfile
35. sorbet-eraser - Versions diffs - 0.5.0 → 0.6.0 - RubyGems - Mend, https://my.diffend.io/gems/sorbet-eraser/prev/0.6.0
36. Persisting data? · Issue #609 - chartdb - GitHub, https://github.com/chartdb/chartdb/issues/609
37. how to generate, write, and host dbt docs free on github pages, https://phillant.com/posts/20260330-dbt-docs/
38. dbt docs generate: Producing Your Project's Documentation, https://npblue.com/data/dbt/dbt-docs-generate/
39. Documentation — Descriptions, dbt docs, and Why It Matters | Topfolio, https://www.topfolio.in/courses/dbt-fundamentals/dbt-fundamentals-m01-building-a-real-dbt-project/documentation-descriptions-dbt-docs-and-why-it-matters
40. About dbt docs commands | dbt Developer Hub, https://docs.getdbt.com/reference/commands/cmd-docs
41. Deploying dbt Docs in Production for Multiple Projects, Self-Hosted, https://medium.com/towardsdev/deploying-dbt-docs-in-production-for-multiple-projects-self-hosted-and-secured-part-1-3-64aa43229329
42. About dbt compile command | dbt Developer Hub,

https://docs.getdbt.com/reference/commands/compile
43. dbt - DataHub, https://docs.datahub.com/docs/generated/ingestion/sources/dbt
44. Getting started - dbt-bouncer - GitHub Pages, https://godatadriven.github.io/dbt-bouncer/v3.8.0/getting_started/
45. Using MetricFlow queries in dbt models - Bolt, https://bolt.eu/en/blog/using-metricflow-queries-in-dbt-models/
46. DBT doesn't throw an error when running dbt docs generate but the, https://stackoverflow.com/questions/71577610/dbt-doesnt-throw-an-error-when-running-dbt-docs-generate-but-the-catalog-json-i
47. Control access to resources with IAM | BigQuery, https://docs.cloud.google.com/bigquery/docs/control-access-to-resources-iam
48. Your dbt Models Are Your Context Layer: Native dbt Integration in, https://www.getwren.ai/post/no-more-modeling-start-asking-dbt-wren
49. Scaling data analytics with software engineering best practices, https://building.nu.com/scaling-data-analytics-with-software-engineering-best-practices/
50. Tools, Types, Formats & Pricing - Architecture Diagram FAQ, https://architecturediagram.ai/faq
51. GitHub - nubank/NuOpenMetadata: OpenMetadata is a unified, https://github.com/nubank/NuOpenMetadata
52. Map Data Catalog API methods to Knowledge Catalog, https://docs.cloud.google.com/dataplex/docs/map-data-catalog-apis
53. View data lineage for Google Cloud systems | Knowledge Catalog, https://docs.cloud.google.com/knowledge-catalog/docs/use-lineage
54. Data Lineage API roles and permissions, https://docs.cloud.google.com/iam/docs/roles-permissions/datalineage
55. Control access with IAM Conditions | BigQuery, https://docs.cloud.google.com/bigquery/docs/conditions
56. Changes to dataset-level access controls | BigQuery, https://docs.cloud.google.com/bigquery/docs/dataset-access-control
57. BigQuery IAM roles and permissions - Google Cloud Documentation, https://docs.cloud.google.com/bigquery/docs/access-control